

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 2 – Pseudocode Writing Task (Algorithm Design)

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO5 – Articulation	Assessment:	Assessment Tool 2 – Pseudocode Writing Task (Algorithm Design)
Weightage:	35% of CIA		
CO5 Statement:	Implement semantic models using predicate logic, word sense disambiguation and validate information retrieval techniques. (BL-5)		
Student Name:	Nirmal Kasi	Reg. No.:	192424248
Section / Batch:	B.Tech AI/DS	Date:	

Code of Conduct

I Nirmal Kasi 192424248 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Nirmal

Instructions

- Draw necessary diagrams such as constraint graphs, coreference chains, discourse trees, or predicate logic representations wherever applicable.
- Generate solutions that fully satisfy all given constraints and provide clear evaluation of the output.
- Answers should be concise, well-structured, and supported by evidence from the given text/scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Reference Resolution, Discourse	Entity Linking Algorithm	Q1
Text Coherence, Discourse Structure	Coherence Analysis Algorithm	Q2
Dialog Acts, Conversational Agents	Intent Recognition Algorithm	Q3
Language Generation, Surface Realization	NLG Pipeline Design	Q4
Machine Translation, Interlingua, Statistical Approaches	Translation Workflow Algorithm	Q5

Annexure A – Pseudocode Writing Task (Algorithm Design)

1. Reference Resolution Using Multiple Constraints

Design an algorithm to perform **Reference Resolution** by identifying pronouns and selecting the most appropriate antecedent using gender, number, recency, and semantic compatibility.

Apply your algorithm to the discourse:

"Meena gave Kavya a laptop because she needed one for her project. She thanked her and started using it."

Perform the following steps:

1. Identify all entities and referring expressions.
2. Generate possible antecedents for each pronoun.
3. Apply gender, number, recency, and semantic constraints.
4. Select the most appropriate antecedent.
5. Generate the resolved discourse.

Finally, explain how your algorithm avoids incorrect resolution when multiple antecedents have similar grammatical properties.

2. Algorithm for Entity Chains and Discourse Coherence

Design an algorithm to identify **entity chains** in a discourse and use them to evaluate text coherence.

Apply your algorithm to the following text:

"Anjali bought a new laptop. The laptop had a powerful processor. She installed Python on it. Later, Anjali used the computer to develop a machine learning application."

Perform the following steps:

1. Identify all entities and referring expressions.
2. Link expressions referring to the same entity.
3. Construct entity chains.
4. Calculate or estimate the continuity of entities across sentences.
5. Determine whether the discourse is coherent.

Finally, explain how breaking an entity chain could reduce discourse coherence.

3. Algorithm for Identifying Discourse Relations

Design an algorithm to identify **discourse relations** between consecutive sentences using discourse markers and contextual meaning.

Apply your algorithm to:

"The server crashed unexpectedly. As a result, users could not access the application. The technical team restarted the server. After that, the system became available again."

Your algorithm should identify relations such as:

- Cause–Effect
- Sequence
- Problem–Solution
- Elaboration

Perform the following steps:

1. Divide the discourse into individual discourse units.
2. Identify explicit discourse markers.
3. Analyze the semantic relationship between units.
4. Assign an appropriate discourse relation.
5. Construct a discourse structure representation.

Finally, justify how the identified relations make the discourse logically coherent.

4. Algorithm for Dialogue Act Classification

Design an algorithm to recognize **Dialogue Acts** in a conversation.

Apply your algorithm to:

User: "Hello, I need help with my assignment."

Agent: "Sure, what problem are you facing?"

User: "I do not understand machine learning."

Agent: "I can explain the basic concepts to you."

Classify each utterance into one of the following:

- Greeting
- Request
- Question
- Inform
- Offer
- Confirmation

Perform the following steps:

1. Preprocess each utterance.
2. Extract relevant linguistic features.
3. Identify the communicative intention.
4. Assign a dialogue act.
5. Generate the dialogue-act sequence.

Finally, explain how dialogue-act recognition helps a conversational agent select an appropriate next response.

5. Algorithm for Dialogue State Tracking

Design an algorithm for a **Conversational Agent** that maintains dialogue state while collecting information from a user.

Apply your algorithm to:

User: "I want to book a flight."

Agent: "Where would you like to travel?"

User: "I want to go to Delhi."

Agent: "From which city?"

User: "From Chennai."

Your algorithm should maintain the following slots:

Departure City

Destination City

Travel Date

Number of Passengers

Perform the following steps:

1. Initialize the dialogue state.
2. Identify information provided in each utterance.
3. Update the corresponding slot.
4. Identify missing information.
5. Generate the next appropriate question.

Finally, explain how dialogue state tracking improves coherence in a multi-turn conversation.

Q1. Reference Resolution Using Multiple Constraints

Given discourse:

“Meena gave Kavya a laptop because she needed one for her project. She thanked her and started using it.”

1. Identify Entities and Referring Expressions

Entities:

- Meena
- Kavya
- Laptop
- Project

Referring expressions:

- she
- one
- her
- She
- her
- it

2. Possible Antecedents

Referring Expression	Possible Antecedents
First she	Meena, Kavya
one	Laptop
her	Meena, Kavya
Second She	Meena, Kavya
Second her	Meena, Kavya
it	Laptop

3. Apply Constraints

The algorithm uses **gender, number, recency, and semantic compatibility** to select the most appropriate antecedent.

First “she”

Sentence:

“Meena gave Kavya a laptop because she needed one for her project.”

Both Meena and Kavya are possible female antecedents.

Using semantic and discourse context, the most natural interpretation is:

she → Kavya

because Kavya is the recipient of the laptop and is likely to need it for her project.

“one”

one → laptop

“One” replaces the noun **laptop**.

First “her”

her → Kavya

The project is associated with the person who needs the laptop.

Second “She”

She → Kavya

The discourse continues with Kavya as the person using the laptop.

Second “her”

her → Meena

Kavya thanked Meena, the person who gave her the laptop.

“it”

it → laptop

Kavya started using the laptop.

4. Final Coreference Chains

Kavya:

Kavya → she → her → She

Meena:

Meena → her

Laptop:

laptop → one → it

5. Resolved Discourse

“Meena gave Kavya a laptop because Kavya needed one for her project. Kavya thanked Meena and started using the laptop.”

6. How the Algorithm Avoids Incorrect Resolution

The algorithm does not depend on only one factor. It combines:

Gender → Number → Recency → Grammatical Role → Semantic Compatibility

If two antecedents have the same gender and number, the system uses **recency, grammatical role, and semantic compatibility** to select the most appropriate antecedent. This reduces incorrect resolution when multiple entities have similar grammatical properties.

Q2. Algorithm for Entity Chains and Discourse Coherence

Given text:

“Anjali bought a new laptop. The laptop had a powerful processor. She installed Python on it. Later, Anjali used the computer to develop a machine learning application.”

1. Identify Entities and Referring Expressions

Entities:

- Anjali
- Laptop
- Processor
- Python
- Computer
- Machine learning application

Referring expressions:

- The laptop
- She
- it
- Anjali
- the computer

2. Link Expressions to the Same Entity

Expression	Entity
Anjali	Anjali
The laptop	Laptop
She	Anjali
it	Laptop
Anjali	Anjali
the computer	Laptop

3. Entity Chains

Person chain:

Anjali → She → Anjali

Laptop chain:

Laptop → The laptop → it → the computer

The laptop remains the main object throughout the discourse.

4. Entity Continuity

Sentence	Main Entity	Reference
1	Anjali, Laptop	Introduced
2	Laptop	Repeated
3	Anjali, Laptop	Pronouns
4	Anjali, Laptop	Repeated and referred to

There is strong continuity because the important entities are repeatedly mentioned or referred to using pronouns and related expressions.

5. Coherence

The discourse is **coherent**.

The logical progression is:

Anjali buys laptop

↓

Laptop has powerful processor

↓

Anjali installs Python on laptop

↓

Anjali uses laptop to develop application

Each sentence continues the topic established in the previous sentence.

6. Effect of Breaking an Entity Chain

If the system incorrectly resolves “**it**” to Python instead of the laptop, the sentence would imply:

“She installed Python on Python.”

This is semantically incorrect.

Similarly, if “**the computer**” were treated as a completely new entity rather than the laptop, the connection between the sentences would become weaker.

Therefore, maintaining entity chains improves **continuity, consistency, and discourse coherence**.

Q3. Algorithm for Identifying Discourse Relations

Given discourse:

“The server crashed unexpectedly. As a result, users could not access the application. The technical team restarted the server. After that, the system became available again.”

1. Divide into Discourse Units

DU1: The server crashed unexpectedly.

DU2: Users could not access the application.

DU3: The technical team restarted the server.

DU4: The system became available again.

2. Identify Discourse Markers

- “As a result” → Cause–Effect
- “After that” → Sequence

3. Analyze Semantic Relationships

DU1 → DU2

Server crashed → Users could not access application

The server crash caused the application to become inaccessible.

Relation = Cause–Effect

DU2 → DU3

Users cannot access application → Team restarts server

The team performs an action to address the problem.

Relation = Problem–Solution

DU3 → DU4

Team restarted server → System became available

The second event occurs after the first.

Relation = Sequence

4. Discourse Structure

DU1: Server crashed

|

| Cause–Effect

↓

DU2: Users cannot access application

|

| Problem–Solution

↓

DU3: Team restarted server

|

| Sequence

↓

DU4: System became available

5. Relation Table

Discourse Units	Relation	Reason
DU1 → DU2	Cause–Effect	Crash caused access failure
DU2 → DU3	Problem–Solution	Restart addresses the problem

Discourse Units	Relation	Reason
DU3 → DU4	Sequence	Availability follows restart

6. Justification

These relations create a logical chain:

Problem → Effect → Solution → Recovery

The discourse is therefore coherent because every sentence has a meaningful relationship with the surrounding sentences. The explicit markers “**As a result**” and “**After that**” make the relationships easier to identify.

Q4. Algorithm for Dialogue Act Classification

Conversation:

User: “Hello, I need help with my assignment.”

Agent: “Sure, what problem are you facing?”

User: “I do not understand machine learning.”

Agent: “I can explain the basic concepts to you.”

1. Preprocess Utterances

Remove unnecessary punctuation and identify important words and sentence patterns.

2. Extract Linguistic Features

Important features include:

- Greeting words
- Question words
- Request expressions
- Statements of information
- Offer expressions
- Confirmation expressions

3. Identify Communicative Intention

Utterance	Intention
“Hello, I need help with my assignment.”	Greeting + Request
“Sure, what problem are you facing?”	Question
“I do not understand machine learning.”	Inform
“I can explain the basic concepts to you.”	Offer

4. Dialogue-Act Sequence

Greeting + Request → Question → Inform → Offer

5. Classification

Speaker	Utterance	Dialogue Act
User	Hello...	Greeting
User	I need help...	Request
Agent	What problem...	Question
User	I do not understand...	Inform
Agent	I can explain...	Offer

6. Importance of Dialogue-Act Recognition

Dialogue-act recognition allows the conversational agent to understand the **purpose of each utterance**.

For example:

User → Request

The system should provide help.

User → Inform

The system should acknowledge the information and respond appropriately.

Agent → Question

The system is requesting additional information.

Therefore, dialogue-act recognition helps the agent choose an appropriate next response and maintain a logical conversation flow.

Q5. Algorithm for Dialogue State Tracking

Given conversation:

User: “I want to book a flight.”

Agent: “Where would you like to travel?”

User: “I want to go to Delhi.”

Agent: “From which city?”

User: “From Chennai.”

1. Initialize Dialogue State

Initially:

Slot	Value
Departure City	Unknown
Destination City	Unknown
Travel Date	Unknown
Number of Passengers	Unknown

2. Process First User Utterance

Input:

“I want to book a flight.”

Detected intent:

Book Flight

No slot values are provided.

State:

Slot	Value
Departure City	Unknown
Destination City	Unknown
Travel Date	Unknown
Number of Passengers	Unknown

The agent asks:

“Where would you like to travel?”

3. Process Second User Utterance

Input:

“I want to go to Delhi.”

Extract:

Destination City = Delhi

Updated state:

Slot	Value
Departure City	Unknown
Destination City	Delhi
Travel Date	Unknown
Number of Passengers	Unknown

The agent asks:

“From which city?”

4. Process Third User Utterance

Input:

“From Chennai.”

Extract:

Departure City = Chennai

Updated state:

Slot	Value
Departure City	Chennai
Destination City	Delhi
Travel Date	Unknown
Number of Passengers	Unknown

5. Identify Missing Information

Still missing:

- Travel Date
- Number of Passengers

The next appropriate question can be:

“What is your travel date?”

After receiving the date, the system can ask:

“How many passengers are travelling?”

6. Final Dialogue State

Intent: Book Flight

Departure City: Chennai
Destination City: Delhi
Travel Date: Unknown
Number of Passengers: Unknown

7. Pseudocode

BEGIN

Initialize dialogue state:

```
Departure = NULL
Destination = NULL
TravelDate = NULL
Passengers = NULL
```

FOR each user utterance:

Extract entities and slot values

```
IF destination is detected:
    Destination = detected destination
```

```
IF departure city is detected:
    Departure = detected departure city
```

```
IF travel date is detected:
    TravelDate = detected date
```

```
IF passenger count is detected:
    Passengers = detected number
```

Check for missing slots

```
IF Destination is NULL:
    Ask for destination
```

```
ELSE IF Departure is NULL:
    Ask for departure city

ELSE IF TravelDate is NULL:
    Ask for travel date

ELSE IF Passengers is NULL:
    Ask for number of passengers

ELSE:
    Confirm booking details
```

END

8. Importance of Dialogue State Tracking

Dialogue State Tracking stores information collected across multiple turns. It prevents the system from repeatedly asking for information that the user has already provided.

It maintains coherence by remembering:

Chennai → Departure

Delhi → Destination

and then collecting the remaining required information.

Thus, it enables the conversational agent to conduct a smooth and logically connected **multi-turn conversation**.

Rubrics for Calculation

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q. No. / Criteria Level	Problem Understanding	Algorithm Design	Use of Control Structures	Pseudocode Structure	Completeness	Sub. Total	Total %
1	4	3	4	4	4	19	97
2	4	4	4	4	4	20	
3	4	4	3	4	4	19	
4	4	4	4	4	4	20	
5	4	4	4	4	3	19	

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____